

Securización mediante JWT

M8: IMPLEMENTACIÓN DE API BACKEND NODE EXPRESS

|AE4: IMPLEMENTAR MECANISMOS DE SEGURIDAD A UN SERVICIO REST DE UN SERVIDOR EXPRESS UTILIZANDO JWT DE ACUERDO AL ENTORNO NODE.JS

Introducción

La seguridad es un aspecto esencial en el desarrollo de servicios REST. Cuando un cliente accede a recursos protegidos, como datos personales o acciones sensibles, es necesario implementar mecanismos de autenticación que garanticen que solo usuarios autorizados puedan interactuar con la API.

En este manual aprenderás a utilizar **JWT (JSON Web Token)** como método de autenticación **stateless** en un servidor Express. Este enfoque te permitirá generar y validar tokens de forma segura, sin necesidad de mantener sesiones en el servidor, facilitando la escalabilidad y simplicidad del backend.

Vas a conocer los componentes internos de un token JWT, cómo se genera, cómo se valida y cómo se protege una ruta utilizando este tipo de autenticación. Además, vas a aplicar buenas prácticas para asegurar tus servicios de manera efectiva en entornos productivos.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender el funcionamiento de JWT como mecanismo de autenticación sin estado (stateless).
- Identificar los componentes principales de un token JWT: header, payload y firma.
- Implementar un endpoint que genere tokens de autenticación utilizando [jsonwebtoken](#).
- Validar y decodificar tokens recibidos en peticiones HTTP.
- Proteger rutas específicas de una API REST usando middleware.
- Aplicar buenas prácticas en el almacenamiento y uso de tokens JWT.

¿Qué es JWT y por qué es importante para la seguridad?

¿Qué es JWT?

JWT (**J**SON **W**eb **T**oken) es un estándar abierto (RFC 7519) que permite transmitir información de forma **segura y compacta** entre dos partes (cliente y servidor) como un **objeto JSON codificado** y firmado digitalmente.

Se utiliza principalmente para:

- Autenticación de usuarios
- Autorización de acceso a recursos protegidos
- Comunicación entre servicios

¿Por qué usar JWT?

JWT permite **autenticar usuarios sin necesidad de almacenar sesiones en el servidor**. Esto es ideal para arquitecturas modernas como SPAs (Single Page Applications) o APIs REST, ya que:

- El servidor no guarda el estado de los usuarios.
- El token viaja en cada solicitud y se puede verificar en cada llamada.
- Se mejora la **escalabilidad** del sistema.

Este modelo se conoce como **stateless authentication**.

¿Cómo funciona un flujo típico con JWT?

1.  El usuario envía sus credenciales (usuario/contraseña).
2.  El servidor verifica las credenciales y **genera un token JWT** firmado con una clave secreta.
3.  El cliente almacena ese token (en `localStorage`, `sessionStorage` o en memoria).
4.  En cada solicitud protegida, el cliente **envía el token en el header Authorization**.

5. 🧠 El servidor valida el token y permite (o no) el acceso a los recursos.

🧠 Diferencia con sesiones tradicionales

Sesiones tradicionales	JWT
El servidor guarda el estado	Stateless, no guarda nada
Requiere almacenamiento (RAM, DB)	Solo clave secreta para verificar
Más difícil de escalar	Escalabilidad sencilla

Estructura de un token JWT (Header, Payload, Signature)

Un token JWT está compuesto por **tres partes** separadas por puntos (.). Cada parte está codificada en **Base64URL** y juntas forman un string como este:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c3VhcmVlIjoieYWRtaW4iLCJpYXQiOiJlY29kZ4ODg4ODg5ImV4cCI6MTY4ODg5MjQ4OH0.dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

1. Header

Contiene información sobre el algoritmo de firma y el tipo de token.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **alg**: algoritmo de firma (ej: HS256, RS256)
- **typ**: tipo de token (siempre "JWT")

📦 Este header es codificado en Base64URL.

2. Payload

Contiene la **información (claims)** que se desea transmitir. Puede incluir:

- Datos personalizados (como el usuario, rol, etc.)

- Claims estándar como:
 - **iat**: fecha de emisión (Issued At)
 - **exp**: fecha de expiración (Expiration)
 - **sub**: sujeto (identificador único)
 - **iss**: emisor

Ejemplo:

```
{  
  "usuario": "admin",  
  "rol": "superuser",  
  "iat": 1688888888,  
  "exp": 1688892488  
}
```

! El payload **no está encriptado**. Está codificado, pero puede ser leído si alguien accede al token.

🧩 3. Signature

Es la **firma digital** que verifica que el token no fue alterado.

Se genera así:

```
HMACSHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  clave_secreta  
)
```

🧠 El servidor usa una **clave secreta** para firmar el token y verificar su integridad en cada solicitud.

🔒 Importante

El **token completo** es lo que se envía en el header de autorización:

```
Authorization: Bearer <TOKEN>
```

- Si alguien modifica el payload o el header, la firma se invalida.

Generación de un token JWT en Express

Para trabajar con JWT en Express vamos a usar el paquete `jsonwebtoken`, que nos permite **crear y verificar tokens** de manera sencilla.

Paso 1: Instalar el paquete

Desde la terminal en tu proyecto:

```
npm install jsonwebtoken
```

Paso 2: Definir una ruta para autenticar y generar el token

Supongamos que queremos autenticar un usuario ficticio y devolverle un token si las credenciales son válidas.

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();

app.use(express.json());

const SECRET_KEY = 'mi_clave_secreta_super_segura';

// Endpoint para autenticación y generación de token
app.post('/login', (req, res) => {
  const { usuario, contraseña } = req.body;

  // Simulación de validación de credenciales
  if (usuario === 'admin' && contraseña === '1234') {
    const payload = {
      usuario: usuario,
      rol: 'admin'
    };

    // Generar el token (con expiración de 1 hora)
    const token = jwt.sign(payload, SECRET_KEY, { expiresIn: '1h' });

    res.json({
      mensaje: 'Autenticación exitosa',
      token: token
    });
  }
});
```

```
});
} else {
  res.status(401).json({ mensaje: 'Credenciales inválidas' });
}
});
```

✅ Resultado esperado

Si enviás un **POST /login** con:

```
{
  "usuario": "admin",
  "contraseña": "1234"
}
```

Vas a recibir:

```
json
CopiarEditar
{
  "mensaje": "Autenticación exitosa",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

🧠 Tip: Nunca compartas tu clave secreta

Guardá la clave en variables de entorno con **dotenv** si estás en producción.

Validación de token y protección de rutas

Una vez que el usuario tiene un token, debe enviarlo en cada solicitud a una ruta protegida. El servidor lo verifica y solo permite el acceso si el token es válido y no ha expirado.

📦 Paso 1: Middleware para validar el token

Vamos a crear un middleware llamado **verificarToken** que valide la existencia y validez del JWT:

```
function verificarToken(req, res, next) {
  const authHeader = req.headers['authorization'];
```

```
// El token viene en el formato: "Bearer <token>"
const token = authHeader && authHeader.split(' ')[1];

if (!token) {
  return res.status(401).json({ mensaje: 'Token no proporcionado' });
}

jwt.verify(token, SECRET_KEY, (err, decoded) => {
  if (err) {
    return res.status(403).json({ mensaje: 'Token inválido o expirado' });
  }

  // Guardamos el payload decodificado en req.usuario
  req.usuario = decoded;
  next();
});
}
```

Paso 2: Proteger una ruta

Ahora aplicamos el middleware a una ruta privada:

```
app.get('/perfil', verificarToken, (req, res) => {
  res.json({
    mensaje: 'Bienvenido a tu perfil',
    datos: req.usuario
  });
});
```

¿Cómo se envía el token desde el cliente?

En Postman o desde el frontend:

```
Authorization: Bearer <tu_token>
```

¿Qué pasa si el token no es válido?

- Si el token está mal formado, expirado o no es proporcionado, el middleware responde con **401** o **403**, según el caso.
- Si el token es válido, se permite el acceso y el contenido del **payload** queda disponible en **req.usuario**.

Buenas prácticas de seguridad con JWT

El uso de JWT es poderoso, pero también puede exponer riesgos si no se implementa correctamente. Acá van las principales buenas prácticas para usar JWT de forma segura y profesional:

1. Nunca compartas la clave secreta

La clave usada para firmar y verificar el token (`SECRET_KEY`) debe mantenerse segura y **nunca debe quedar en el código fuente**. Usá variables de entorno:

```
require('dotenv').config();
const SECRET_KEY = process.env.JWT_SECRET;
```

2. Establecé expiraciones cortas para los tokens

Usá la opción `expiresIn` al firmar el token. Ejemplo:

```
jwt.sign(payload, SECRET_KEY, { expiresIn: '1h' });
```

Esto limita el tiempo en el que un token robado puede usarse.

3. Validá siempre el token en cada solicitud

Nunca confíes en un token sin verificarlo primero. Usá `jwt.verify()` siempre, incluso en rutas internas.

4. No guardes información sensible en el payload

Recordá que el payload del JWT **no está encriptado**, solo codificado. Cualquier persona con el token puede leerlo. No pongas contraseñas, tarjetas de crédito, etc.

5. Usá HTTPS siempre

Si transmitís tokens por HTTP, un atacante puede interceptarlos fácilmente. Asegurate de que tu servidor y frontend estén usando **HTTPS** para proteger la comunicación.

6. Podés usar refresh tokens

Para sesiones más largas, podés emitir un token de acceso (corto) y un refresh token (más largo) que permita renovar el acceso sin volver a loguearse.

Con estas prácticas, tu autenticación con JWT será más **segura, robusta y escalable** 

Cierre

En esta lección aprendiste a implementar seguridad en un servicio REST usando **JWT (JSON Web Token)**, una solución moderna y escalable para gestionar autenticación sin necesidad de sesiones en el servidor.

Comenzamos con los conceptos teóricos: qué es JWT, por qué es importante, cómo funciona internamente y cómo se integra en un flujo de autenticación. Luego, construimos paso a paso:

- Un endpoint de login que genera un token con `jsonwebtoken`
- Un middleware que valida y decodifica el token en cada solicitud protegida
- Rutas privadas que solo pueden ser accedidas si se proporciona un token válido

Por último, revisamos buenas prácticas para asegurar que la implementación sea robusta y segura, especialmente en ambientes productivos.

Con este conocimiento, estás en condiciones de proteger tus APIs y controlar el acceso de usuarios de forma confiable.

Referencias

- Auth0. (s.f.). What is JSON Web Token? <https://auth0.com/learn/json-web-tokens/>
- Express.js. (s.f.). Documentación oficial de Express. <https://expressjs.com/es/>
- jsonwebtoken. (s.f.). NPM. <https://www.npmjs.com/package/jsonwebtoken>
- JWT.io. (s.f.). Introducción y herramientas. <https://jwt.io/>
- Mozilla. (s.f.). Códigos de estado HTTP. MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTTP/Status>
- Jones, M., Bradley, J., & Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). IETF. <https://datatracker.ietf.org/doc/html/rfc7519>

¡Muchas gracias!

Nos vemos en la próxima lección

